

Practice Exercise 15: Digital Signature Challenge-Response with PKI

1. Goal

In this exercise, you will build a secure login system using **Digital Signatures**. To stop hackers from faking their identity, we will use a **Certificate Authority (CA)** to verify public keys.

Important Rule: Do NOT calculate RSA math yourself (e.g., $\text{pow}(m, d, n)$). You must use the Python cryptography library for real-world security.

The Actors:

1. **Certificate Authority (CA):** A trusted entity. **Its Public Key is known by everyone.**
2. **Prover (Client):** The user trying to log in.
3. **Verifier (Server):** The system checking the login. It trusts **only** the CA.

2. Technical Requirements

You must use these exact settings to match the test data:

- **Library:** cryptography (Install: `pip install cryptography`).
- **Key Type:** RSA 2048-bit.
- **Hash Algorithm:** SHA-256.
- **Signature Padding: PSS** (Probabilistic Signature Scheme).
 - *MGF:* MGF1.
 - *Salt Length:* Max length.
- **Data Formats:**
 - **Keys:** PEM format (String starts with `-----BEGIN PUBLIC KEY...`).
 - **Signature:** Hexadecimal string.

3. Certificate Format (JSON)

A certificate is a JSON object containing user info and the CA's signature.

```
{
  "info": {
    "id": "Alice",
    "public_key": "-----BEGIN PUBLIC KEY-----\nMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ
8AMIIBCgKCAQEA..."
  },
  "signature": "a1b2c3d4..."
}
```

- **info:** The data to be signed (User ID + User Public Key).
- **signature:** The CA's signature of the info block.

4. Coding Task

Implement these classes:

A. Class CryptoHelper

Write static helper functions to wrap the library:

- `load_pem_public_key(pem_str)`: Loads a key from a string.
- `load_pem_private_key(pem_str)`: Loads a key from a string.
- `sign(private_key, data)`: Returns a Hex signature (Using PSS + SHA256).
- `verify(public_key, data, signature_hex)`: Returns True or False.

B. Class Prover

- **Init:** Stores the user's Private Key and Certificate.
- **Function solve_challenge(nonce):** Signs the random string (nonce) received from the Server.

C. Class Verifier

- **Init(ca_public_key_pem):**
 - This class MUST be initialized with the **CA Public Key** (provided in the Test Dataset).
 - Store this key as the "Root of Trust".
- **Function verify_session(prover, nonce):**
 1. Get the Certificate from the Prover.
 2. **Verify Certificate:** Use the stored **CA Public Key** to verify the signature on the Certificate. (If invalid, return False immediately).
 3. Extract the **User's Public Key** from the Certificate.
 4. **Verify Challenge:** Verify the Prover's signature on the nonce using the extracted User Public Key.

5. Test Data & Scenarios

You must use the **Test Data Generator** script (see Appendix) to generate a JSON dataset. Your code should verify two scenarios using that data.

The Dataset Structure

When you run the generator, it will output a JSON object. Pay attention to CA_PUBLIC_KEY:

```
{
  "CA_PUBLIC_KEY": "-----BEGIN PUBLIC KEY-----\nMI... (This is the Root of Tr
```

```
ust) ...",

"ALICE_PRIVATE_KEY": "...",
"ALICE_CERTIFICATE": { ... },

"BOB_PRIVATE_KEY": "...",
"BOB_PUBLIC_KEY": "..."
}
```

Scenario 1: Successful Login (Alice)

- **Setup:** Initialize Verifier with dataset["CA_PUBLIC_KEY"].
- **Prover:** Initialize Prover with ALICE_PRIVATE_KEY and ALICE_CERTIFICATE.
- **Execution:** verify_session(alice_prover, "nonce_123").
- **Expected: TRUE.**

Scenario 2: Impersonation Attack (Bob)

- **Context:** Bob stole Alice's Certificate but only has his own Private Key.
- **Prover:** Initialize Prover with BOB_PRIVATE_KEY but use ALICE_CERTIFICATE.
- **Execution:** verify_session(bob_prover, "nonce_456").
- **Expected: FALSE.** (Certificate is valid, but Challenge Response is invalid).

6. Submission Guidelines

- **File Name:** ChallengeResponse.py.
- **Content:** Classes (CryptoHelper, Prover, Verifier) + main() function.
- **Data:** Include the generated JSON data as a variable in your code.

Appendix: Test Data Generator

Run this script **once** to get your keys.

```
import json
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import rsa, padding

# --- HELPER FUNCTIONS ---
def generate_key_pair():
    """Generates an RSA 2048-bit key pair."""
    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048,
    )
    public_key = private_key.public_key()
    return private_key, public_key
```

```

def key_to_pem_str(key, is_private=False):
    """Converts a Key object to a PEM string."""
    if is_private:
        pem = key.private_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PrivateFormat.PKCS8,
            encryption_algorithm=serialization.NoEncryption()
        )
    else:
        pem = key.public_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PublicFormat.SubjectPublicKeyInfo
        )
    return pem.decode('utf-8')

def sign_data(private_key, data_dict):
    """Signs a dictionary using PSS + SHA256."""
    data_str = json.dumps(data_dict, sort_keys=True)
    signature = private_key.sign(
        data_str.encode('utf-8'),
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )
    return signature.hex()

# --- MAIN GENERATOR ---
if __name__ == "__main__":
    print("--- GENERATING TEST DATA FOR EXERCISE 15 ---")

    # 1. Create CA (Certificate Authority)
    ca_priv, ca_pub = generate_key_pair()

    # 2. Create User (Alice)
    alice_priv, alice_pub = generate_key_pair()

    # 3. Create Attacker (Bob)
    bob_priv, bob_pub = generate_key_pair()

    # 4. Issue Certificate for Alice
    alice_info = {
        "id": "Alice",
        "public_key": key_to_pem_str(alice_pub)
    }
    alice_signature = sign_data(ca_priv, alice_info)

    alice_cert = {

```

```

    "info": alice_info,
    "signature": alice_signature
}

# --- OUTPUT ---
dataset = {
    # *** THIS IS THE ROOT OF TRUST FOR THE VERIFIER ***
    "CA_PUBLIC_KEY": key_to_pem_str(ca_pub),

    "ALICE_PRIVATE_KEY": key_to_pem_str(alice_priv, is_private=True),
    "ALICE_CERTIFICATE": alice_cert,

    "BOB_PRIVATE_KEY": key_to_pem_str(bob_priv, is_private=True),
    "BOB_PUBLIC_KEY": key_to_pem_str(bob_pub)
}

print("\n[DATASET JSON START]")
print(json.dumps(dataset, indent=2))
print("[DATASET JSON END]")

print("\n--- INSTRUCTIONS ---")
print("1. Copy the JSON output above.")
print("2. Paste it into your 'ChallengeResponse.py' file.")
print("3. Ensure your Verifier class uses 'CA_PUBLIC_KEY'.")

```